

ann-java

A complete, beginner-friendly guide to a from-scratch approximate nearest-neighbour search library in Java — what it does, how every part works, and what it measured.

Covers HNSW, IVF-PQ, SIMD distance kernels, the benchmark protocol, 360 measured configurations across two datasets, and a comparison with FAISS.

Repository: github.com/ankur94532/ann-java

Written to be readable with no prior knowledge of vector search.

How to read this document

This guide assumes you can read a little code and nothing else. Every term is defined the first time it appears. You do not need to know what a vector database, a quantizer, or SIMD is — those are all explained here.

Boxes like this define jargon

Whenever a technical word appears for the first time, a box like this explains it in plain language before the text uses it again.

Boxes like this are the important idea

If you read nothing else in a section, read these.

Boxes like this are mistakes and surprises

This project made real errors and found real surprises. They are marked rather than hidden, because they are usually the most instructive part.

The document is organised so you can stop at any point and still have a complete picture at that level of depth:

- **Parts 1–3** — the problem, the data, and the rules. No code.
- **Parts 4–7** — how each algorithm actually works, with code.
- **Parts 8–10** — how it was measured, what the numbers said, what went wrong.
- **Parts 11–12** — every file in the repository, and how to run it yourself.

Contents

The problem

- 1.1 What "nearest neighbour" means
- 1.2 Why the obvious method is too slow
- 1.3 The bargain: approximate search
- 1.4 Recall — measuring "good enough"

The data

- 2.1 SIFT1M and GIST1M
- 2.2 The .fvecs file format
- 2.3 One flat array, not an array of arrays
- 2.4 Ground truth, and a surprise in it

The rules

- 3.1 Why freeze a protocol
- 3.2 What is fixed

Measuring distance fast

- 4.1 Squared Euclidean distance
- 4.2 The scalar loop
- 4.3 SIMD and the Java Vector API
- 4.4 The hidden dependency chain
- 4.5 Results

HNSW: search by walking a graph

- 5.1 The intuition
- 5.2 The layers
- 5.3 How a search works
- 5.4 When the search stops
- 5.5 How the graph is built
- 5.6 Neighbour selection — the key decision
- 5.7 The three parameters

Making HNSW fast

- 6.1 The naive version
- 6.2 Five changes, measured
- 6.3 What the table teaches

IVF-PQ: search by compressing

- 7.1 Inverted files
- 7.2 k-means clustering
- 7.3 Product quantization
- 7.4 Residuals
- 7.5 The lookup table trick
- 7.6 A layout mistake worth 5x
- 7.7 The three parameters

How it was measured

- 8.1 The harness
- 8.2 JMH and JFR
- 8.3 The FAISS baseline

What the numbers said

- 9.1 SIFT1M
- 9.2 GIST1M — what breaks
- 9.3 The three gradients

What went wrong

- 10.1 Four real mistakes
- 10.2 Three failed optimizations

Every file in the repository**Running it yourself****Glossary**

1. The problem

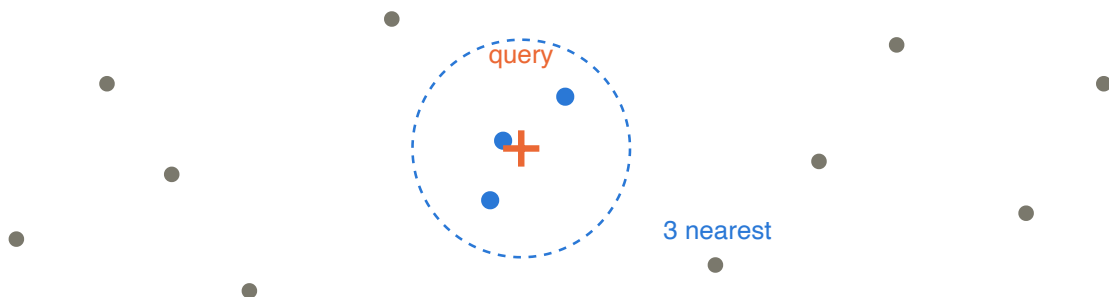
1.1 What "nearest neighbour" means

Modern software turns things into *vectors*: lists of numbers that describe something. A photograph becomes 960 numbers. A sentence becomes 768 numbers. A song becomes 128 numbers. The numbers are produced by some model, and the useful property is that **similar things get similar numbers**.

Vector

An ordered list of numbers, like `[0.31, -1.2, 0.04, ...]`. The count of numbers is the **dimension**. A 128-dimensional vector is a list of 128 numbers. You can picture a 2-dimensional vector as a point on a page and a 3-dimensional one as a point in a room; beyond that the picture stops helping, but the arithmetic is identical.

Once your things are points in space, "find me photos similar to this one" becomes a geometry question: **which stored points are closest to this new point?** That is nearest-neighbour search, and it is the operation underneath image search, recommendation, de-duplication, and the retrieval step of most systems that put a language model on top of a document collection.



The task, drawn in two dimensions. The cross is a new item; the ringed points are its three nearest stored items. Real vectors have 128 or 960 dimensions, but the question is the same one.

1.2 Why the obvious method is too slow

The obvious method works perfectly: measure the distance from your query to every single stored vector, keep the ten smallest. This is called **brute force** or **exact search**, and it is always right.

It is also unusably slow at scale. This project measured it: on one million 128-dimensional vectors, one query takes about **14 milliseconds**. That sounds fast until you notice it means roughly 70 queries per second per CPU core. A service handling ten thousand queries a second would need 140 cores just for search.

The core difficulty

Exact search costs time proportional to the number of stored vectors. Ten times more data means ten times slower. Every technique in this guide exists to break that proportionality — to answer in time that grows like $\log n$ rather than n , or to make each comparison so cheap that n stops mattering.

1.3 The bargain: approximate search

The way out is to give up being always right. An **approximate** nearest neighbour index returns *probably* the nearest neighbours, thousands of times faster. For almost every real use — showing similar photos, retrieving relevant documents — getting 9 of the 10 true nearest is indistinguishable from getting all 10.

This project implements the two families that dominate practice:

FAMILY	IDEA IN ONE SENTENCE	TRADES
HNSW (graph)	Build a network of "who is near whom", then walk it downhill from a random start.	Very fast and very accurate; uses a lot of memory.
IVF-PQ (quantized)	Group vectors into clusters, then <i>compress</i> each vector to a few bytes.	Tiny memory footprint; noticeably less accurate.

1.4 Recall — measuring "good enough"

If an index is allowed to be wrong, you need a number for how wrong. That number is **recall**.

recall@10

Ask the index for the 10 nearest neighbours. Compare its 10 answers with the 10 *true* nearest neighbours. Recall@10 is the fraction that overlap, averaged over every query. If the index returns 9 of the 10 correct items, recall is 0.9. Order does not matter — only which items came back.

So every result in this project is a pair: **recall** (how right) and **latency** (how fast). Neither alone means anything. An index that answers in one microsecond with recall 0.2 is useless, and so is one with recall 0.999 that takes a second. The interesting object is the *curve* connecting them, because every index has a dial that trades one for the other.

2. The data

2.1 SIFT1M and GIST1M

Benchmarking needs data everyone agrees on, or results cannot be compared between projects. This project uses two standard public datasets from the TEXMEX corpus, both containing exactly one million vectors.

DATASET	VECTORS	DIMENSIONS	QUERIES	WHAT THE NUMBERS DESCRIBE
SIFT1M	1,000,000	128	10,000	A small patch of an image: 16 spatial cells $\times$ 8 gradient directions each.
GIST1M	1,000,000	960	1,000	A whole image at once — a "gist" descriptor summarising the entire scene.

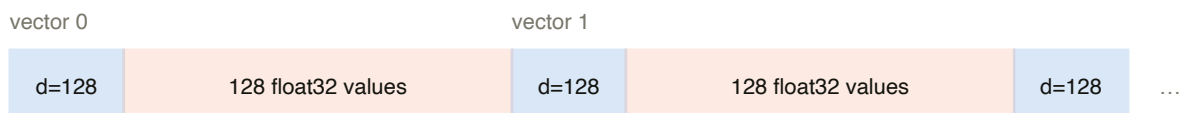
The pairing is deliberate. SIFT is the easy case at 128 dimensions; GIST is the hard case at 960. Running the identical code on both is what lets this project say which findings are about the algorithms and which are about dimensionality.

The curse of dimensionality

In high-dimensional space, distances between points become strangely uniform — everything is roughly equally far from everything else. That flattens the "downhill" signal any search relies on. It is the reason GIST is hard and it shows up repeatedly in Part 9.

2.2 The .fvecs file format

The datasets ship in a format so simple it has no header at all. A `.fvecs` file is just records, back to back:



4 bytes 512 bytes

The file size divided by $(4 + 4d)$ gives the number of vectors — nothing else records it.

The `.fvecs` layout. Every record repeats the dimension, which is redundant but makes the format self-checking: the reader verifies every record declares the same width.

The matching `.ivecs` format is identical but holds 32-bit integers instead of floats. It is used for the ground truth: row i lists the ids of the true nearest neighbours of query i .

2.3 One flat array, not an array of arrays

Here is the first design decision that matters for speed. The natural Java representation of a million 128-dimensional vectors is `float[1_000_000][128]` — a million separate little arrays. This project instead uses **one** array of 128 million floats, with vector i living at positions `[i×128, (i+1)×128)`.

Why flat is faster

In Java, `float[][]` is an array of *references*. Reading vector 500's third number means: load the outer array, follow a pointer to vector 500's array, then read inside it. That is two dependent memory accesses, and the million inner arrays are scattered all over the heap.

With one flat array, the address is pure arithmetic — $500 \times 128 + 3$ — no pointer to follow, and consecutive vectors sit in consecutive memory, which is exactly what CPU caches and prefetchers are built for.

2.4 Ground truth, and a surprise in it

To measure recall you need the true answers. Both datasets ship them, but this project recomputed them from scratch with its own brute-force search and compared — because if the file reader has a bug, every subsequent number is silently wrong, and a loader bug looks exactly like a good approximate index.

The check produced a genuine surprise.

Perfect recall is impossible — the answer is not unique

On SIFT, query 93 has two different stored vectors — 196106 and 274922 — at *exactly* the same distance, 42192.0. Which one is "the 10th nearest neighbour"? Both are. The shipped file picked one, this project's search picked the other, and neither is wrong.

That is not a rounding artefact. SIFT components are whole numbers from 0 to 255, so every squared distance is a whole number below 16,777,216 — and a 32-bit float represents those exactly. The tie is real.

646 of the 10,000 SIFT queries have such ties. **So the highest recall anyone can score on SIFT under this protocol is 0.999440, not 1.0.** GIST has the same problem for a different reason (Part 9), with a ceiling of 0.999200.

This changed how correctness is checked. Comparing *ids* is meaningless when *ids* are not unique. The project compares **distances** instead, and one-sidedly: finding something *closer* than the official answer is never an error. Under that rule, the brute-force search reproduces both datasets exactly.

3. The rules

3.1 Why freeze a protocol

Benchmarks are easy to cheat without meaning to. If you decide *after* seeing results which configurations to report, which query set to use, or how to define recall, you will drift toward whatever flatters your code. Nobody does this deliberately; it happens by a hundred small reasonable-seeming choices.

So this project wrote `PROTOCOL.md` **before implementing a single index** and did not change it afterwards. Every number in the results comes from that document's rules.

3.2 What is fixed

RULE	SETTING, AND WHY
Task	Find the 10 nearest neighbours (<code>k=10</code>) under squared Euclidean distance.
Recall	Set overlap with the true 10, averaged over all queries. Order ignored.
Query set	The complete shipped set — 10,000 for SIFT, 1,000 for GIST. Never a sample.
Latency	Single-threaded, one query at a time. Warm-up pass discarded, then three measured passes, median reported.
Build	Single-threaded on <i>both</i> sides. Slower than either could go, but the only setting where "mine took X and FAISS took Y" means anything.
Memory	Index structures only, excluding the raw vectors — so a graph is compared against a graph, not against a graph plus half a gigabyte.
Parameters	The exact grid to sweep, listed in advance so it cannot be narrowed later.

The rule that mattered most

Late in the project, a checkpoint target turned out to be unreachable, and the tempting fix was to widen the frozen parameter grid until something passed. The protocol forbade it. The result was reported as **missed** instead, with an explanation of why the target was self-contradictory — which is a far more useful finding than a passing grade would have been. See Part 7.7.

4. Measuring distance fast

Every index in this project ultimately does one thing over and over: measure how far apart two vectors are. Get that wrong and nothing else matters; make it fast and everything speeds up. So it comes first.

4.1 Squared Euclidean distance

The distance between two vectors is the ordinary straight-line distance, extended to many dimensions. For vectors a and b :

$$\text{distance}^2 = (a_0 - b_0)^2 + (a_1 - b_1)^2 + \dots + (a_{127} - b_{127})^2$$

Notice this project never takes the square root. Square root is monotonic — if $x^2 < y^2$, then $x < y$ — so it changes no ordering. For a search that only needs to know which is nearer, taking it is a wasted operation per comparison, and there are billions of comparisons.

4.2 The scalar loop

The direct translation is four lines:

```
float sum = 0f;
for (int i = 0; i < dim; i++) {
    float d = a[aOff + i] - b[bOff + i];
    sum += d * d;
}
return sum;
```

Correct, obvious, and the reference every faster version is checked against. It handles one number at a time — hence "scalar".

4.3 SIMD and the Java Vector API

SIMD

Single Instruction, Multiple Data. Modern CPUs have wide registers that hold several numbers at once and instructions that operate on all of them in one step. On the machine used here (an Apple M4 Pro), a register holds **4 floats**, so one instruction can do four subtractions. In principle that is $4\times$ fewer instructions.

Java exposes this through the incubating **Vector API**. The same computation becomes:

```
FloatVector acc = FloatVector.zero(SPECIES);
for (int i = 0; i < bound; i += LANES) { // LANES = 4 here
    FloatVector diff = FloatVector.fromArray(SPECIES, a, aOff + i)
        .sub(FloatVector.fromArray(SPECIES, b, bOff + i));
    acc = diff.fma(diff, acc); // acc += diff * diff, fused
}
float sum = acc.reduceLanes(VectorOperators.ADD); // add the 4 lanes together
```

`fma` is a *fused multiply-add*: it multiplies and adds in one instruction with a single rounding, which is both faster and slightly more accurate than doing them separately.

4.4 The hidden dependency chain

Here is where the project found something genuinely non-obvious. Measured speedups:

KERNEL	D=128	SPEEDUP	D=960	SPEEDUP
scalar	53.2 ns	1.00×	714.2 ns	1.00×
SIMD, 1 accumulator	12.8 ns	4.16×	183.8 ns	3.89×
SIMD, 4 accumulators	13.3 ns	4.00×	70.2 ns	10.17×

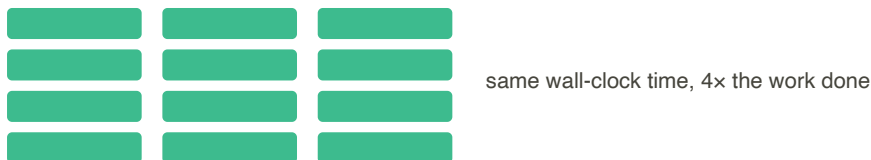
The 4.16× at d=128 is expected — four lanes, roughly four times faster. But **10.17× is impossible on a four-lane machine**. You cannot get more than 4× from having 4 lanes. So something else must be going on, and it must be that the *scalar baseline* is unusually slow at 960 dimensions.

It is, and the reason is a dependency chain. In `sum += d*d`, each addition needs the result of the previous addition. The CPU cannot start addition number 7 until number 6 has finished. Floating-point addition takes several clock cycles to complete, so the loop runs at one addition per *latency*, no matter how many arithmetic units are sitting idle.

One accumulator — each step waits for the previous one



Four accumulators — four independent chains, issued back to back



Why four accumulators help. Java may not reorder a floating-point sum (it would change the answer's last bits), so the single-accumulator loop is stuck waiting. Splitting into four independent running totals lets four additions be in flight at once.

The general lesson

A speedup *larger than the lane count* is never extra parallelism in the SIMD code — it is a serial dependency in the baseline that the SIMD version was allowed to break. At d=960 the four-accumulator kernel wins 4× from lanes and another 2.6× from breaking the chain. At d=128 it wins only the lanes, and the extra accumulators actually cost 0.5 ns because setting up and combining four running totals cannot be amortised over just 32 steps.

4.5 Results

A second measurement worth knowing: scanning a large block of vectors sequentially runs at the same speed whether the data is in the fastest cache or out in main memory (75.2 million vectors/second either way, at d=128). The CPU's prefetcher sees the predictable pattern and fetches ahead perfectly. **This means a**

sequential scan is limited by arithmetic, not by memory — which turns out to matter a lot in Part 6, because a graph walk is *not* sequential and does not get that benefit.

5. HNSW: search by walking a graph

5.1 The intuition

HNSW stands for *Hierarchical Navigable Small World* graph. Strip the name and the idea is simple: instead of comparing the query to every vector, build a network in which each vector knows a few of its near neighbours, then *walk* the network toward the query.

The walk is greedy. Stand on some vector, look at its neighbours, step to whichever is closer to the query, repeat. When no neighbour is closer, you have arrived. If the network is well built, this touches a few thousand vectors instead of a million.

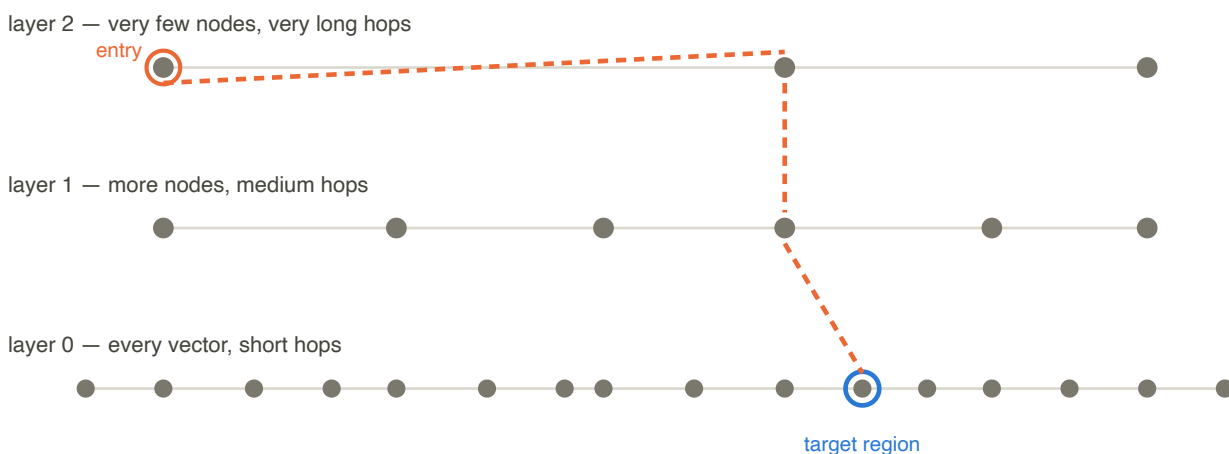
Graph, node, edge, degree

A **graph** is dots joined by lines. Each dot is a **node** — here, one stored vector. Each line is an **edge** — "these two are near each other". A node's **degree** is how many edges it has. The whole design problem of HNSW is choosing *which* edges to keep, since you cannot afford many.

5.2 The layers

Pure greedy walking has a flaw: if you start far away, you take an enormous number of small steps. HNSW fixes this with **layers**, borrowing the trick from skip lists.

Every vector lives in layer 0. A random few also live in layer 1, a random few of those in layer 2, and so on — each layer holding roughly $1/M$ of the one below. Upper layers are sparse, so their edges span long distances.



Descent: long hops at the top narrow the region, short hops at the bottom refine it.

The hierarchy. A search enters at the single top node, takes a few long strides, and drops a layer each time it can no longer improve — arriving at layer 0 already in the right neighbourhood. This is what makes the cost grow like $\log n$ instead of n .

A node's top layer is drawn randomly when it is inserted, using `floor(-ln(uniform(0,1)) × mL)` with $mL = 1/\ln(M)$. That specific formula makes each layer about M times smaller than the one below, which is what produces the logarithmic behaviour.

5.3 How a search works

A query runs in two phases.

Phase 1 — descend. Start at the single entry point in the top layer. Walk greedily to the closest node, drop one layer, repeat. This is cheap: a handful of distance computations per layer, done only to find a good starting position.

Phase 2 — beam search at layer 0. Greedy walking alone gets stuck in local minima, so at the bottom layer the search keeps a *set* of candidates rather than one position. Two collections do the work:

- a **frontier** — nodes discovered but not yet explored, kept ordered so the nearest is always taken next;
- a **result set** — the best `efSearch` answers found so far, kept ordered so the *worst* of them is instantly available.

```
while (frontier is not empty) {
    nearest = frontier.removeClosest();
    if (results are full AND nearest is farther than the worst result)
        break;                                     // <-- the termination condition
    for (neighbour of nearest) {
        if (already visited) continue;
        d = distance(query, neighbour);
        if (results not full OR d < worst result distance) {
            frontier.add(neighbour, d);
            results.add(neighbour, d);               // drops the worst if over capacity
        }
    }
}
```

5.4 When the search stops

That `break` deserves its own section, because it is the entire approximation.

The frontier is always popped nearest-first. So the moment the nearest unexplored candidate is *farther away* than the worst answer already held, every remaining candidate is also farther. The search assumes that following edges from those can only lead to points that are not much nearer — and stops.

What `efSearch` actually controls

It is tempting to read `efSearch` as "how many results to return". It is not — that is `k`, and `k` is 10 throughout this project while `efSearch` ranges from 16 to 512.

`efSearch` is **how much worse than the current best a candidate may be and still be worth exploring**. A bigger beam keeps a worse "worst result", which raises the bar the termination test must clear, which lets the search push through a ridge of slightly-worse nodes to reach a better valley behind it. That is the entire recall/latency dial.

The assumption — that unexplored branches cannot hold anything much better — is a property of a well-built graph, not a theorem. It is exactly why recall is not 1.0, and why a badly built graph (Part 5.6) fails in a way no amount of `efSearch` repairs.

5.5 How the graph is built

Insertion reuses the search. To add a new vector:

1. Draw its top layer randomly.
2. Descend greedily from the entry point down to that layer — same as phase 1 of a search.
3. At each layer from there down to 0, run the *same* beam search, but with width `efConstruction` instead of `efSearch`. The results are candidate neighbours.
4. Choose which candidates to actually connect to (next section).
5. Add edges **both ways**, and if that pushes an existing node over its degree limit, re-select its neighbours.

Step 5 matters. An edge that only points from new to old is useless: a search arriving at the old node would never discover the new one. But adding the back-edge is what can overflow an existing node's budget, and how you then prune is the same decision as step 4.

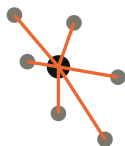
5.6 Neighbour selection — the key decision

A node can only afford M edges ($2M$ at layer 0). The beam search offered it perhaps 200 candidates. Which do you keep?

Option A — keep the M nearest. Obvious, and much worse than it looks.

Option B — the pruning heuristic. Walk candidates nearest-first and keep a candidate e only if no already-kept neighbour r is closer to e than e is to the node itself.

A — keep the M nearest



no edge leaves the cluster —
a search that lands here is trapped
measured: 3,783 of 4,000 nodes unreachable

B — the pruning heuristic



long edge survives
redundant near edges are dropped,
freeing budget for an unexplored direction
measured: 0 unreachable

The two strategies on clustered data. Nearest- M spends every edge inside the local cluster, because the nearest candidates are all cluster-mates. The heuristic notices those edges are redundant — you can already reach them via each other — and spends one on the direction nothing else covers.

Read the rejection test as a redundancy test

If some kept neighbour r is closer to candidate e than e is to node q , then a search at q can reach e by hopping to r first. Spending one of q 's few edges on e buys a shortcut that already exists. Spending it on a candidate nothing covers buys reachability that did not.

This project implements both and measured the difference on real data. On a 4,000-node clustered test, nearest-M left **3,783 nodes unreachable** from the entry point, and recall fell to 0.67 against the heuristic's 0.98. The failure is not gradual — it is a hard ceiling, because the needed edge does not exist and no search parameter conjures it.

5.7 The three parameters

PARAMETER	WHEN IT APPLIES	WHAT RAISING IT DOES
M	build	More edges per node. Better connected, more memory (M=8 → 77 MiB, M=32 → 260 MiB), slower search per hop.
efConstruction	build	Wider beam while inserting, so each node sees more of the graph before choosing edges. Better graph, longer build, <i>no</i> query cost.
efSearch	query	The recall/latency dial. Changeable any time without rebuilding — which is why one build serves a whole curve.

6. Making HNSW fast

A correct HNSW and a fast HNSW are different programs. This project wrote the correct one first, kept it, and then made five changes one at a time — measuring each.

The rule that makes this table trustworthy

Both implementations draw random layers from the same seeded generator in the same order and order candidates by the same total ordering, so they build a **byte-identical graph** — and a test asserts exactly that. Without it, a "35% faster" row could just as easily be a slightly worse graph that searches less thoroughly, and the whole table would mean nothing. Recall is identical to six decimal places in every row.

6.1 The naive version

The reference implementation uses the obvious Java data structures: `ArrayList<Integer>` for neighbour lists, `HashSet<Integer>` to remember which nodes have been visited, and `PriorityQueue<Candidate>` for both heaps. On SIFT1M it builds in 779 seconds and answers a query in $447\ \mu\text{s}$ (p95) at $\text{ef}=64$.

6.2 Five changes, measured

#	CHANGE	P95 @ EF=64	BUILD	MEMORY
0	naive reference	$447.5\ \mu\text{s}$	779.3 s	494.8 MiB
1	flat <code>int[]</code> neighbour arenas	$357.0\ \mu\text{s}$	603.7 s	137.9 MiB
2	versioned visited stamps	$182.3\ \mu\text{s}$	360.4 s	137.9 MiB
3	primitive <code>long[]</code> heaps	$160.4\ \mu\text{s}$	282.6 s	137.9 MiB
4	split traversal from distances	$159.2\ \mu\text{s}$	283.1 s	137.9 MiB
5	software prefetch	$117.0\ \mu\text{s}$	220.2 s	137.9 MiB

Cumulative: $3.8\times$ faster queries, $3.5\times$ faster builds, $3.6\times$ less memory — with byte-identical results.

Step 1 — flat arenas (–20%)

Each node's neighbour list was an `ArrayList<Integer>`. Reading one edge meant: load the `ArrayList` object, follow a pointer to its backing array, follow another pointer to an `Integer` box, read the `int` inside. Four dependent memory accesses, on four unrelated cache lines, to deliver four bytes.

Now layer 0 is one big `int[]` where node n 's list starts at $n \times \text{stride}$ — arithmetic instead of a pointer chase, and the whole list arrives in one or two cache lines.

The surprise: this is the change most people would bet on, and it was the small one. The memory saving was dramatic ($494.8 \rightarrow 137.9$ MiB — the boxed version spent about 21 bytes to store 4), but the speed gain was only 20%.

Step 2 — versioned visited stamps (–49%, the big one)

A search must not examine the same node twice, so it keeps a set of visited nodes. That was a fresh `HashSet<Integer>` per layer per query, and every one of the $\sim 1,300$ nodes a query touches cost an object allocation, a hash, and a probe into scattered memory.

The replacement is one `int` per node holding the number of the search that last touched it:

```
boolean visit(int node) {
    if (stamp[node] == generation) return false;    // already seen
    stamp[node] = generation;
    return true;
}
// clearing the whole set for the next query:
generation++;
```

Testing membership is one array read and one compare. Clearing it is incrementing a counter rather than touching a million entries.

The surprise: half of everything remaining was going into a hash set that never held more than a few thousand items and looks completely harmless in the source. Its cost scales with exactly the quantity `efSearch` controls, so it grew precisely where it hurt.

Step 3 — primitive heaps (–12%)

Both heaps stored objects. Now a (distance, id) pair is packed into a single `long`: the distance in the top 32 bits, transformed so that ordinary integer comparison of the longs gives the right ordering, and the id in the bottom 32 bits.

```
long packed = ((long) sortableBits(distance) << 32) | (id & 0xffffffffL);
```

A comparison becomes one `long` compare of values already in registers, the heaps are plain `long[]`, and the search loop allocates nothing at all. Packing the id into the low bits also makes the ordering *total*, which is what lets this implementation produce a byte-identical graph rather than merely a similar one.

Step 4 — split traversal from distances (no effect)

Instead of interleaving "read a neighbour id, compute its distance", the loop first collects all unvisited neighbour ids, then computes their distances. The idea was to let several memory fetches overlap.

It did nothing — $159.2\ \mu\text{s}$ against 160.4 , which is noise. The distance computations had no dependency on one another in the original loop either, so the processor was already overlapping them. The optimization aimed at a stall that was not happening.

Step 5 — software prefetch (–27%)

A profiler put 54.8% of query time in the distance kernel. But arithmetic alone should not cost that: at `ef=64` a query does 1,323 distance computations in $125\ \mu\text{s}$, which is 52 ns each, against 13.3 ns for the same kernel on cached data. **Three quarters of the cost of a distance was waiting for memory** — each one reads 512 bytes from a random place in half a gigabyte, and unlike a sequential scan, a graph walk gives the hardware prefetcher nothing to predict.

Java has no prefetch instruction. What works is reading one float from each pending vector into a field the compiler cannot prove is unused, before the distance loop runs. Those reads are independent, so they queue up in the memory system together while the first distance is still being computed.

Step 4 was not a failure — it was half of step 5

Splitting traversal from distance computation is what makes the *complete set* of pending addresses known before any distance is computed, which is the precondition for issuing their fetches together. On its own it looked like dead weight and was the obvious thing to revert. A table that kept only the changes that paid off immediately would have thrown away the enabling half of its biggest win.

6.3 What the table teaches

- **Intuition about performance is unreliable.** The predicted big win was 20%; the unsuspected one was 49%.
- **Measure the thing, not a proxy.** "54.8% in the kernel" sounds like a compute problem; dividing by the number of distance computations revealed a memory problem, and those want opposite fixes.
- **Keep the failures.** Step 4's zero result is the reason step 5 exists.
- **Changes that scale with a parameter tell you what they touched.** Step 3's benefit grew from -8% at $ef=16$ to -17% at $ef=512$, which identifies it as per-candidate work that beam width multiplies.

7. IVF-PQ: search by compressing

HNSW is fast but needs the full vectors in memory — 488 MiB of raw data plus a 138 MiB graph. IVF-PQ answers a different question: *what if the data does not fit?* It indexes the same million vectors in **19.7 MiB** — $25\times$ smaller than the raw data — by throwing away most of each vector.

The name is two techniques stacked: **IVF** (inverted file) narrows *where* to look, **PQ** (product quantization) shrinks *what* is stored.

7.1 Inverted files — narrowing the search

Split the million vectors into `nlist` groups by proximity — say 1,024 groups of about a thousand each. Record the centre of each group. To search: compare the query to the 1,024 centres, pick the `nprobe` closest groups, and scan only those.

At `nlist=1024` and `nprobe=8` you examine roughly 8,000 vectors instead of 1,000,000. The risk is obvious: if a true neighbour sits in a group you did not open, it is lost. Raising `nprobe` opens more groups — more accurate, slower. That is IVF's dial.

7.2 k-means clustering

The groups come from **k-means**, which is worth understanding because it is the most expensive step in the whole project.

k-means

Pick k starting centres. Then repeat: (1) assign every point to its nearest centre; (2) move each centre to the average of the points assigned to it. Each round fits slightly better. Twenty-five rounds is the standard stopping point.

The cost is step (1). With a million training points and 4,096 centres over 25 rounds, that is **102 billion distance computations** — around 18 minutes, and by far the longest single operation this project runs.

7.3 Product quantization — the compression

Now the clever part. A 128-dimensional float vector is 512 bytes. PQ turns it into `m` bytes — 16, say — a $32\times$ reduction.

Cut the vector into `m` equal pieces. For each piece *position*, run k-means over that piece across the whole dataset to learn **256 representative values** — a codebook. Now any vector's piece can be replaced by the number (0–255) of the closest representative. That number fits in one byte.

one vector, 128 floats = 512 bytes



each piece → index of its nearest of 256 codebook entries

41	203	7	118	92	250	33	176	88	14	201	60	129	45	238	99
----	-----	---	-----	----	-----	----	-----	----	----	-----	----	-----	----	-----	----

16 bytes — 32× smaller

16 independent 256-entry codebooks describe $256^{16} \approx 10^{38}$ distinct points, using only $16 \times 256 \times 8$ stored floats. A single codebook would need 10^{38} entries.

Product quantization. The compression comes from the codebooks being a *product*: combining 16 independent choices of 256 options describes an astronomically large set of points with a tiny amount of stored data.

What is lost is exactness. A code names a *cell*, not a point. Distances computed from codes are distances to the cell's representative, and that error puts a ceiling on recall that no amount of `nprobe` can lift.

7.4 Residuals — why the codes are worth more than they look

IVF-PQ does not quantize the vector. It quantizes the **residual**: the vector minus its group's centre.

Encoding x directly asks 256-entry codebooks to cover the whole dataset. Encoding $x - \text{centre}(x)$ asks them to cover only the spread *within* one group, which is much smaller — so the same 16 bytes buy a far finer approximation. The cost appears at search time and is discussed next.

7.5 The lookup table trick

Here is the mechanism that makes scanning compressed vectors fast. Because squared distance is a sum over independent pieces, the distance from a query to *any* encoded vector is the sum of `m` table lookups.

For one query: for each of the 16 piece-positions, compute the distance from the query's piece to all 256 codebook entries. That is a 16×256 table, built once. Then each stored vector costs 16 lookups and 15 additions — **no arithmetic on the vector itself, and the vector is never needed**. That is why the index does not store them.

"Asymmetric" distance computation

The *query* is not compressed — only the stored vectors are. Both could be, which would make lookups even cheaper, but the query would then carry its own approximation error on top. Keeping the query exact costs nothing at search time and roughly halves the error.

The residual trick has a price here: the query's residual is $\text{query} - \text{centre}$, which differs for every group opened. So the table must be rebuilt `nprobe` times per query, not once.

7.6 A layout mistake worth 5×

Measure both halves before optimizing either

IVF-PQ search was 5.5× slower than FAISS. The natural assumption is that scanning is slow. So the two halves were timed separately: **table construction 895 μs (62%)**, **list scanning 546 μs (38%)**. The table build was the problem, not the scan.

The cause was how codebooks were laid out in memory. Stored centroid-by-centroid, building the table means 256 separate calls to a distance function over pieces of 8 dimensions (or 4, or 2). At those lengths a SIMD kernel is nearly all setup cost — about 4 ns of overhead for 1 ns of work — and an IVF-PQ query pays it $nprobe \times m \times 256$ times.

The fix was not a faster small kernel. It was to *never call one*. Storing the codebooks **transposed** — all 256 codewords' values for one dimension laid out consecutively — lets the table be computed by vectorising across the *codeword* axis instead: broadcast one query number, subtract 4 codewords at once, accumulate across the piece's dimensions in a register, store 4 finished distances.

M	BEFORE	AFTER	SPEEDUP
8	10.93 μs	2.24 μs	4.9×
16	13.51 μs	2.71 μs	5.0×
32	25.71 μs	4.72 μs	5.4×
64	51.17 μs	6.15 μs	8.3×

End to end this took a query from 1,461 μs to 694 μs — **2.1× faster**, with identical recall. The lesson generalises: the fix for a slow small kernel is often to restructure so it is not called, rather than to make it faster.

7.7 The three parameters, and a target that cannot be hit

PARAMETER	WHEN	EFFECT
<code>nlist</code>	build	Number of groups. More means shorter lists and less scanning, but a costlier coarse search and a higher chance a neighbour sits in an unopened group.
<code>m</code>	build	Bytes per code. Sets both compression ratio and the recall ceiling.
<code>nprobe</code>	query	Groups opened per query. The recall/latency dial.

Measured recall ceilings on SIFT1M, at the largest `nprobe` :

CODE SIZE	BEST RECALL@10	COMPRESSION
m=16 (16 bytes)	0.5741	24.8×
m=32 (32 bytes)	0.7380	14.0×
m=64 (64 bytes)	0.8979	7.5×

The checkpoint that could not be met

The project plan asked for **recall above 0.80 with at least $8\times$ compression**. Those look like two independent requirements. They are not — they are two ends of one dial. Reaching 0.80 needs 64-byte codes, and 64-byte codes are $7.5\times$ compression. Since m must divide 128, there is nothing available between 32 and 64.

FAISS shows the same ceilings at the same code sizes, so this is a property of 8-bit product quantization, not of this implementation. The target was reported as **missed**, with both endpoints of the frontier shown — rather than widening the frozen parameter grid until something passed.

8. How it was measured

8.1 The harness

Every reported number comes from the same code path, so nothing can differ between runs by accident. For each configuration the harness:

1. builds the index once and records build time and memory;
2. runs a **warm-up pass** whose timings are thrown away;
3. runs three **measured passes** over the complete query set;
4. reports the median of the three.

Why warm up?

Java starts by interpreting bytecode and only compiles a method to machine code after seeing it run enough times. The first thousand queries can be an order of magnitude slower than the steady state. Measuring them would report the JIT compiler's warm-up, not the algorithm. The warm-up is a *separate* pass so that no query is dropped from the measured set — which matters for GIST, whose whole query set is only 1,000 long.

Results are written to CSV, one row per configuration, and the runner is **resumable**: it reads back what is already there and skips it, so a seven-hour sweep can be interrupted without losing work.

8.2 JMH and JFR

Two more tools, for two different questions.

JMH (Java Microbenchmark Harness) answers "how fast is this one function?" Timing small Java operations naively gives garbage — the compiler deletes code whose result is unused, and warm-up effects dominate. JMH handles all of it. It produced the kernel table in Part 4.

JFR (Java Flight Recorder) answers "where does the time actually go?" It samples the call stack thousands of times a second and reports which methods were executing. It is what identified the memory stall behind step 5.

A profiler can lie to you by default

The command that prints JFR recordings truncates every stack trace to **five frames** unless told otherwise. At five frames, a sample taken during index construction and one taken during a query look identical — both are just `[kernel, searchLayer, ...]`. The first profile taken in this project confidently attributed the two phases to each other. The numbers looked entirely plausible, which is the dangerous part. The analysis script now refuses to report quietly on a truncated recording.

8.3 The FAISS baseline

FAISS

Facebook AI Similarity Search — the standard open-source library for this problem, written in C++ with hand-tuned SIMD over roughly a decade. It is the reference every implementation is measured against.

A Python script runs FAISS under identical conditions: same datasets, same k , same recall definition, same machine, same warm-up rule, one query per call, and `omp_set_num_threads(1)` to match the single-threaded protocol. It emits the same CSV columns, so both sides concatenate into one table with no reconciliation step.

Two subtleties were needed to make the comparison honest:

- FAISS's HNSW index serialises a full copy of the raw vectors alongside its graph, so its reported size was 625.9 MiB against this project's 137.9. Subtracting the 488.3 MiB of vectors — which the protocol requires anyway — gives 137.6 MiB. Left uncorrected, the memory plot would have shown a $4.5\times$ gap that does not exist.
- Its parameters had to be set to match rather than left at defaults. FAISS's `IndexIVFPQ` defaults to 10 clustering rounds where the standalone default is 25, and its k-means seeds from random points where this project used a smarter method. Both were aligned, and the seeding difference was measured as an ablation rather than assumed harmless.

9. What the numbers said

9.1 SIFT1M — 128 dimensions

TARGET	INDEX	CONFIGURATION	RECALL@10	LATENCY	FAISS	SIZE
≥ 0.90	HNSW	M=16, efC=200, ef=32	0.9160	53 μ s	60 μ s	138 MiB
≥ 0.99	HNSW	M=16, efC=200, ef=128	0.9920	165 μ s	201 μ s	138 MiB
max	HNSW	M=32, efC=400, ef=512	0.9994	732 μ s	732 μ s	260 MiB
max	IVF-PQ	nlist=4096, m=32, nprobe=64	0.7380	649 μ s	268 μ s	36.5 MiB
—	brute force	—	0.9994	~ 14 ms	—	—

HNSW reaches **0.9994** — **the exact ceiling** the tie analysis established, at $19\times$ the speed of brute force. At the more practical 0.9920 it is $85\times$ faster.

The two families barely compete. HNSW needs 138 MiB of graph *plus* the 488 MiB of raw vectors, because it computes real distances during the search. IVF-PQ's 36.5 MiB is the entire index. If the data fits in memory, use the graph; if it does not, the quantizer is not a compromise, it is the only option that runs.

9.2 GIST1M — 960 dimensions, and what breaks

Identical code, identical parameter grid, $7.5\times$ more dimensions.

BEST RECALL@10	SIFT1M	GIST1M	CHANGE
HNSW, this project	0.9994	0.9884	−1.1 points
HNSW, FAISS	0.9994	0.9905	−0.9 points
IVF-PQ, this project	0.7380	0.2821	−45.6 points
IVF-PQ, FAISS	0.7347	0.2804	−45.4 points

The two families fail completely differently

HNSW degrades *gracefully*: it still lands within 1.1 points of the ceiling and pays about $5\times$ the latency to get there. Given enough `efSearch`, the answer is still findable.

IVF-PQ *collapses* — from three quarters of the true neighbours to barely a quarter — and no `nprobe` recovers it. FAISS loses the same 45 points, which is the check that matters: this is product quantization failing, not this code.

Why: dimensions per byte, not dimensions

A PQ code is `m` bytes whatever the dataset. So each byte's 256 representatives must cover a piece of `dim/m` dimensions:

AT M=32	DIMENSIONS PER BYTE	256 REPRESENTATIVES IN THAT SPACE
SIFT1M	4	dense — about four divisions along each axis
GIST1M	30	hopeless — 256 points cannot tile 30 dimensions at any useful resolution

Nothing about 960 dimensions is intrinsically hard for an inverted file. What is hard is being asked to describe 30 dimensions with one byte. The protocol holds the compression ratio fixed across datasets, so *resolution* is what gives.

Failures concentrate on unusual queries

Bucketing queries by how isolated they are — the distance to their own 10th true neighbour — shows recall is not lost evenly:

	DENSEST TENTH	SPARSEST TENTH	SPREAD
SIFT1M	0.997	0.933	6.4 points
GIST1M	0.942	0.681	26.1 points

The concentration is four times stronger at 960 dimensions. A GIST index reporting 0.77 mean recall is returning 0.94 for typical queries and 0.68 for unusual ones. **If the queries that matter are the unusual ones, the average was the wrong number to have looked at.**

Neighbour selection changes character

Recall lost by using nearest-M instead of the pruning heuristic:

	EFSEARCH	SIFT LOSS	GIST LOSS
	16	0.096	0.125
	64	0.068	0.182
	256	0.027	0.175
	512	0.018	0.163

On SIFT the damage *shrinks* as the beam widens — those edges were shortcuts, and searching harder takes the long way to the same place. On GIST it does not shrink: quadrupling the beam four times over recovers nothing. Those edges were the *only* route, and no beam width fixes an edge that does not exist.

At low dimension the heuristic is an optimization. At high dimension it is what keeps the graph connected at all.

Orphans

A node with no incoming edge cannot be returned by any search, at any setting — nothing points at it. On SIFT: zero queries affected. On GIST: **10 of 1,000**. That is a hard recall floor that no tuning removes, and it appears only at high dimension.

9.3 The three gradients

Against FAISS, this project's HNSW is faster on SIFT at all 36 configurations and slower on GIST at low degree. Read as a single ratio that looks contradictory. It is not.

GIST, EF	M=8	M=16	M=32
16	1.49	1.35	1.07
64	1.47	1.28	0.98
512	1.30	1.26	0.82

Ratio of this project's latency to FAISS's. Below 1.0 means this project is faster. On SIFT the same ratio runs 0.63–0.90 everywhere.

Three consistent trends explain both datasets with one model:

- **More dimensions** → **FAISS gains**. More arithmetic per candidate while bookkeeping per candidate is unchanged. The distance kernel is FAISS's advantage.
- **More efSearch** → **this project gains**. More candidates pass through the visited stamps and the heaps. That bookkeeping is this project's advantage — exactly what steps 2 and 3 bought.
- **More M** → **this project gains**. More pending neighbours per hop for the step-5 prefetch to fetch together, and at 960 dimensions each cache miss costs 3,840 bytes.

Why two datasets were needed

Splitting a performance difference into "kernel" and "bookkeeping" cannot be done by varying `M` alone — `M` changes distance work, hop count, and cache behaviour simultaneously. Varying *dimension* at fixed `M` and `efSearch` moves the kernel/bookkeeping ratio while leaving graph structure and beam mechanics essentially alone. Running both datasets supplied the control that neither could alone.

Three checks the result had to survive

Beating FAISS is not a credible outcome for a from-scratch implementation, so it was treated as a broken comparison until it survived:

1. **Graph sizes match**. 137.6 MiB against 137.9 at `M=16` — 0.2% apart, within 0.7% at every `M`. Two independent implementations filling the same degree budget.
2. **Search work matches**. Distance computations per query: 518 against FAISS's 514 at `ef=16`; 6,942 against 6,781 at `ef=512`. **This project computes ~2% more distances and is still faster** — which also explains its slightly higher recall.
3. **FAISS is not a crippled build**. The installed package reports `OPTIMIZE_ARM_NEON` with the full instruction set available.

All three passed, so the difference is execution efficiency and nothing else. The result is still explicitly scoped: it was measured on an ARM machine where both sides get 4 SIMD lanes. On an x86 machine with AVX-512, FAISS's hand-written code gets 16 lanes against the same 4, and the comparison would plausibly reverse.

10. What went wrong

This section exists because a project that reports only its successes is not a record of what happened. Every item here is documented in the repository itself.

10.1 Four real mistakes

A claimed win over FAISS that was not real

Early results showed this project's IVF-PQ beating FAISS on recall at every setting. It was comparing **this project on 10,000 queries against FAISS on 2,000**. Recall varies enough between query subsets to manufacture a difference the size of the one being claimed. On matched sets, FAISS is very slightly ahead. The README carries a retraction paragraph permanently — a silent table correction would have hidden the more useful lesson.

An estimate wrong by 250×

Validating GIST with the simple distance kernel was projected to take thirteen hours, which justified adding a faster code path. It takes **48 seconds**. The error: multiplying the number of dimensions into the *count* of distance computations, when the dimension is already inside the per-computation cost. The faster path was kept — it forced a useful cross-check into existence — but the protocol records that the reason for adding it was wrong.

A profiler reporting confident nonsense

Described in Part 8.2: five-frame stack truncation made build and query samples indistinguishable, and the resulting attribution was wrong while looking entirely reasonable.

Two CSV defects found by using the tooling, not reading it

The index name column contained unquoted commas — `hnsw(M=16,efC=200,ef=64)` — which silently shifts every later column for any reader. And FAISS's HNSW memory included the 488 MiB of raw vectors it serialises alongside the graph, which would have drawn a 4.5× memory gap that does not exist. Both were caught by scripts that *consumed* the files, not by review.

10.2 Three failed optimizations

ATTEMPT	REASONING	RESULT
Four accumulators in the PQ scan	The same dependency-chain fix that wins 2.6× in the distance kernel.	Nothing — 6.84 μ s against 6.855. The caller's loop over codes already supplied all the parallelism; the distance kernel had no outer loop to hide behind. Reverted.
Cache-blocking k-means	Reuse data while it is in cache instead of re-reading it.	Nothing — 1119.8 s against 1102.4. The loop already ran at 93 million distances/second, <i>above</i> the rate for cached data, because the point being assigned stays in cache across all centroids. There was no memory traffic to remove. Reverted.
A brute-force kernel comparison	Time FAISS's flat scan to test whether its distance kernel is slow.	Discarded before running. FAISS's flat scan uses a different, batched code path from the one its graph search uses. It would have measured the wrong thing and answered nothing.

What the failures had in common

All three came from applying a correct principle to a situation that did not have the problem the principle solves. Dependency chains are real, cache blocking is real, and neither was the bottleneck. The only way to tell is to measure the specific thing before changing it — which is why the IVF-PQ work timed table construction and scanning separately before touching either, and found the problem was in the half nobody would have guessed.

11. Every file in the repository

Top level

PATH	WHAT IT IS
<code>README.md</code>	The main write-up: results, plots, analysis, reproduction steps.
<code>PROTOCOL.md</code>	The frozen benchmark rules. Written before any index existed.
<code>build.gradle</code>	Build config: Java 21 toolchain, the Vector API flag, JMH source set, JFR switch.
<code>gradlew</code> , <code>gradle/</code>	The Gradle wrapper, so a fresh clone builds with no Gradle installed.

Source — `src/main/java/io/shashwat/ann/`

`io/` — reading the data

<code>VecsReader.java</code>	Parses <code>.fvecs</code> / <code>.ivecs</code> . Streams in whole-record chunks; validates every record's declared width.
<code>VectorDataset.java</code>	Holds <code>n</code> vectors of <code>dim</code> dimensions in one flat <code>float[]</code> .
<code>IntDataset.java</code>	The same for integer rows — used for ground truth.
<code>Datasets.java</code>	Where the files live and how to load each part.

`distance/` — the kernels

<code>ScalarDistance.java</code>	Plain loops. The reference everything is checked against.
<code>SimdDistance.java</code>	Vector API versions, single- and four-accumulator.
<code>Distance.java</code>	The facade every index calls; picks the kernel once at startup so the branch folds away.
<code>Metric.java</code>	L2 or inner product, with the "lower is better" convention.

index/ — the indexes

<code>VectorIndex.java</code>	The shared interface. Search writes into caller-supplied arrays so the timed path allocates nothing.
<code>BruteForceIndex.java</code>	Exact search — the oracle everything is validated against.
<code>HnswIndex.java</code>	The naive, obviously-correct HNSW. Kept as the reference and as the optimization table's baseline.
<code>FastHnswIndex.java</code>	The optimized HNSW. Same algorithm, byte-identical graphs, 3.8× faster.
<code>HnswConfig.java</code> , <code>Hnsw.java</code>	Parameters, and the interface both implementations share.
<code>NeighbourSelection.java</code>	The two edge-selection strategies, with the reasoning for each.
<code>GraphDiagnostics.java</code>	Counts orphans, isolated nodes, degree statistics.
<code>IvfPqIndex.java</code>	Inverted file with product-quantized residuals.
<code>IvfPqConfig.java</code>	Its parameters.
<code>KMeans.java</code>	Lloyd's algorithm with two seeding strategies.
<code>ProductQuantizer.java</code>	Codebook training, encoding, and the transposed lookup-table kernel.
<code>Packing.java</code>	Packs (distance, id) into one <code>long</code> with order-preserving bit tricks.
<code>BoundedMaxHeap.java</code>	Keeps the best k. Its root is the <i>worst</i> kept item, so "is this worth keeping?" is one comparison.
<code>LongMinHeap.java</code>	The growable search frontier.

bench/ — measurement

<code>BenchHarness.java</code>	The protocol in code: warm-up, three passes, median, percentiles, memory.
<code>Recall.java</code>	recall@k, defined once for everything.
<code>Measurement.java</code>	One result row, and its CSV encoding.
<code>SweepCommand.java</code>	The full parameter sweep. Resumable.
<code>OracleCommand.java</code>	Validates exact search against shipped ground truth.
<code>HnswCommand.java</code> , <code>IvfPqCommand.java</code>	Single-index runs for development.
<code>AnalysisCommand.java</code>	Per-query analysis: recall, neighbourhood distance, orphan counts.
<code>BenchData.java</code> , <code>CsvSink.java</code>	Data loading and CSV output.

Scripts — `scripts/`

<code>download_sift.sh</code> , <code>download_gist.sh</code>	Fetch the datasets.
<code>setup_python.sh</code>	Creates the virtualenv for FAISS and plotting.
<code>faiss_bench.py</code>	The FAISS baseline under the identical protocol.
<code>plot_results.py</code>	The three curves, light and dark.
<code>plot_failures.py</code>	Recall by neighbourhood-sparsity decile.
<code>hnswn_dis_compare.py</code>	Counts distance computations on both sides — the test that explained the FAISS gap.
<code>jfr_hotspots.py</code>	Turns a JFR recording into a hot-method report; warns on truncated stacks.
<code>opt_table.py</code> , <code>jmh_table.py</code>	Generate the documentation tables from raw results.
<code>repair_csv_quoting.py</code> , <code>repair_faiss_memory.py</code>	One-off repairs for files written before two bugs were fixed.

Documentation and results — `docs/`

<code>kernels.md</code>	The distance-kernel microbenchmarks and what they mean.
<code>hnswn_optimization.md</code>	The six-row optimization table with reasoning for every row.
<code>analysis.md</code>	The GIST high-dimensional failure analysis.
<code>results/*.csv</code>	Every measurement — 360 configurations. Plots regenerate from these.
<code>plots/</code>	Generated charts, light and dark variants.

12. Running it yourself

Requires a JDK 21 installation and about 7 GB of disk for both datasets. Gradle downloads itself.

```
# get the code and the data
git clone https://github.com/ankur94532/ann-java
cd ann-java
./scripts/download_sift.sh          # ~168 MB
./scripts/download_gist.sh          # ~2.6 GB download, 5.8 GB unpacked

# build and run the tests (104 of them)
./gradlew build
./gradlew test

# check the loader against ground truth – start here, everything depends on it
./gradlew run --args="oracle --dataset sift"

# one HNSW index, swept across the recall/latency dial
./gradlew run --args="hns w --m 16 --efc 200 --ef 16,32,64,128,256,512"

# one IVF-PQ index
./gradlew run --args="ivfpq --nlist 1024 --m 16 --nprobe 1,4,8,16,32,64"

# the full sweep – hours, and resumable if interrupted
./gradlew run --args="sweep --dataset sift"

# distance-kernel microbenchmarks
./gradlew jmh -PjmhArgs="DistanceBenchmark"

# profile a run
./gradlew run -Pjfr=true --args="hns w --m 16 --efc 200 --ef 64"
jfr print --stack-depth 64 --events jdk.ExecutionSample build/ann.jfr > samples.txt
python3 scripts/jfr_hotspots.py samples.txt --under BenchHarness.measure

# the FAISS comparison and the plots
./scripts/setup_python.sh
./venv/bin/python scripts/faiss_bench.py --dataset sift --csv docs/results/faiss-sift1m.csv
./venv/bin/python scripts/plot_results.py docs/results/*sift1m.csv --dataset SIFT1M --out docs/p
```

The `--add-modules jdk.incubator.vector` flag that the Vector API needs is handled by the build files. Every CSV behind every published number is committed, so plots and tables regenerate without re-running anything.

13. Glossary

TERM	MEANING
ANN	Approximate Nearest Neighbour. Search that is usually right and far faster than exact.
beam search	Exploring with a set of candidates rather than a single position, so a wrong turn is recoverable.
brute force	Comparing against every stored vector. Always correct, always slow.
cache line	The unit memory moves in, typically 64 bytes. Reading one byte fetches all 64, so layout decides speed.
codebook	The 256 representative values PQ learns for one slice of the vector.
degree	How many edges a graph node has.
dimension	How many numbers are in a vector. 128 for SIFT, 960 for GIST.
efConstruction	Beam width used while building an HNSW graph. Costs build time, not query time.
efSearch	Beam width used while querying. The recall/latency dial.
FAISS	Facebook's C++ similarity-search library; the standard baseline.
FMA	Fused multiply-add. Multiply and add in one instruction with one rounding.
fvecs	The dataset file format: a 4-byte width followed by that many floats, repeated.
ground truth	The true nearest neighbours, computed exhaustively, used to score everything.
HNSW	Hierarchical Navigable Small World graph. The layered-graph index.
inverted file (IVF)	Grouping vectors by cluster so a query only scans nearby groups.
JIT	Just-In-Time compiler. Turns Java bytecode into machine code once a method runs often enough.
JMH / JFR	Java's microbenchmark harness / its low-overhead profiler.
k	How many neighbours to return. 10 throughout this project.
k-means	Clustering by repeatedly assigning points to the nearest centre and recentring.
latency	Time for one query. Reported as mean and 95th percentile.
M	HNSW's edge budget per node. Layer 0 gets 2M.
nlist / nprobe	Number of IVF groups / how many a query opens.
orphan	A node with no incoming edge — unreachable by any search at any setting.
p95	The latency 95% of queries beat. Catches the slow tail a mean hides.
Pareto frontier	The configurations not beaten on both axes at once — the only ones worth plotting.
PQ	Product Quantization. Compressing a vector to m bytes by slicing and coding each slice.
prefetch	Starting a memory fetch before the value is needed, so the wait overlaps other work.
recall@k	Fraction of the true k nearest that the index actually returned.

residual	A vector minus its cluster centre. What IVF-PQ actually compresses.
SIMD	One instruction operating on several numbers at once. 4 floats on this machine.
Vector API	Java's interface to SIMD instructions. Still an incubating feature in Java 21.

Every number in this document was measured on an Apple M4 Pro (10 performance + 4 efficiency cores, 48 GiB) running macOS 26.6 with Temurin OpenJDK 21.0.9 and faiss-cpu 1.15.0, under the protocol in `PROTOCOL.md`. The raw results are committed in `docs/results/`.